Object Oriented in JavaScript

Mastering Object Literals, "this", and Constructor Functions

1. Overview & Why This Matters

The Big Picture

Imagine you are building a video game with 1,000 enemy zombies. If you write out the code for every single zombie one by one, your code will be millions of lines long and impossible to manage. **Object Oriented Programming (OOP)** is a coding style that solves this. Instead of building 1,000 zombies manually, you create one single "Zombie Factory" (a blueprint). Then, you just press a button 1,000 times to stamp out new zombies.

JavaScript is technically a "Prototype-based" language, but it provides OOP syntax so we can build these blueprints easily.

The Mental Model: The Blueprint vs. The House

Think of OOP like building a neighborhood:

- **The Class / Constructor Function:** This is the architect's paper **Blueprint**. You cannot live inside a blueprint, and the blueprint doesn't have a physical front door. It just contains the *instructions* on how to build a house.
- **The Instance (The Object):** This is the actual physical **House** built on the street. You *can* walk into it. You can use the blueprint to build 50 different houses (instances). They all look similar, but House A can have a red door, and House B can have a blue door.

2. Deep-Dive Core Concepts

Concept 1: The Object Literal (The Base Unit)

An object literal is the most basic way to group data together. You literally write out the object using curly braces `{}`. It is great for making **one** specific thing, like your personal user profile.

Concept 2: The `this` Keyword

The word `this` confuses many beginners. **Definition:** `this` simply means "the current context" or "the object I am currently standing inside."

Analogy: If you are standing inside your own house and say "paint **this** walls red", you mean your house. If you walk into your neighbor's house and say "paint **this** walls red", you mean the neighbor's house. `this` changes depending on which object is currently calling it.

Concept 3: The Constructor Function

A Constructor Function is our "Zombie Factory". It is a special function whose only job is to stamp out new, fresh objects based on a set of rules. By convention, we always capitalize the first letter of a Constructor Function (e.g., `function User()` instead of `function user()`).

3. Step-by-Step Code Walkthroughs

Example 1: The Object Literal & "this"

Here we create a single user. We want the `getUserDetails` function to print the username. If we just type `username`, JavaScript will look outside the object and crash. We must use `this.username` to say "look inside THIS object".

```
const user = { // Properties (Data) username: "hitesh", loginCount: 8, signedIn: true, // Method (A function inside an object) getUserDetails: function() { // Without 'this', JS doesn't know which username you mean! console.log(`Got user details from database: ${this.username}`); // To see what 'this' actually is, we can log it: console.log(this); // Prints the entire 'user' object } } user.getUserDetails();
```

Example 2: The Constructor Function & "new" Keyword

What if we want 50 users? Writing 50 object literals is terrible. We build a Constructor Function. Notice how we use parameters to receive data, and `this` to attach that data to the newly built object.

```
// 1. The Blueprint (Capitalized name) function User(username, loginCount, isLoggedIn) { // Left side: The variable that belongs to the new object // Right side: The data passed into the function parameters this.username = username; this.loginCount = loginCount; this.isLoggedIn = isLoggedIn; this.greeting = function() { console.log(`Welcome ${this.username}`); } // We technically don't need to write 'return this', // the 'new' keyword does it for us automatically! return this; } // 2. Using the 'new' keyword to stamp out instances const userOne = new User("hitesh", 12, true); const userTwo = new User("ChaiAurCode", 11, false); console.log(userOne.username); // Prints "hitesh" console.log(userTwo.username); // Prints "ChaiAurCode"
```

Under the Hood: The 4 Magic Steps of the `new` Keyword

When you type the word `new`, JavaScript automatically does 4 things behind the scenes:

1. **Empty Object Creation:** A brand new, completely empty object `{}` is created in memory.
2. **Constructor Function Called:** The `User()` function is called, and it packs up all your arguments ("hitesh", 12, true).
3. **The "this" Binding:** The `this` keyword is forcefully bound to the newly created empty object. So `this.username = "hitesh"` injects data straight into that new object.
4. **Automatic Return:** The fully built object is returned to your variable (e.g., `userOne`).

4. Common Beginner Pitfalls & "Gotchas"

⚠️ The Missing "new" Keyword Trap

The most common mistake beginners make is calling a constructor function like a normal function, completely forgetting the `new` keyword.

```
// ❌ THE WRONG WAY (Forgetting 'new') const userOne = User("hitesh", 12, true); const userTwo = User("sagar", 5, false); // What happens? // 1. Because there is no 'new', an empty object is NEVER created. // 2. 'this' defaults to the Global Window Object! // 3. userTwo completely overwrites all of userOne's data! console.log(userOne); // undefined! // ✅ THE CORRECT WAY const userOne = new User("hitesh", 12, true); const userTwo = new User("sagar", 5, false); // Now they are safely stored in separate memory containers.
```

💡 Arrow Functions and "this"

Never use an Arrow Function `() => {}` inside an object if you need to use the `this` keyword. Arrow functions are "stateless"—they do not have their own `this` context. If you use `this` inside an arrow function, it looks *outside* the object into the global window, which causes bugs.

5. Practical Cheat Sheet Summary

- **Object Literal:** Use `{}` when you only need exactly ONE instance of an item (like a configuration file).
- **Constructor Function:** Use `function Name() {}` when you need a blueprint to create multiple copies of an item (like Users, Enemies, or Products). Always capitalize the name.
- `this`**:** Refers to the current object executing the code.
- `new` **keyword:** Always required when creating instances from a blueprint. It creates a new memory box, binds `this` to that box, and returns the finished object.
- **instanceof:** A built-in JS tool to check if an object was made by a specific blueprint. (e.g., `userOne instanceof User` returns `true`).

6. Comprehensive Knowledge Check

Test your understanding of the lecture with these 5 conceptual questions.

Q1: If you console.log the `this` keyword in an empty, global Node.js file (outside of any function), what does it output?

Answer: It outputs an empty object `{}`. (Note: If you do this in a browser console, it outputs the Global `Window` object. Node and Browsers handle the global context slightly differently).

Q2: What is the primary difference between a Constructor Function and an Instance?

Answer: The Constructor Function is the *Blueprint* (it contains the instructions). The Instance is the actual *Object* created using that blueprint (the physical house).

Q3: You wrote `function Car(brand) { this.brand = brand; }` and then called `const myCar = Car("Toyota");`. What major bug did you just cause?

Answer: You forgot the `new` keyword! Because `new` was missing, JS did not create a new object container. Instead, it attached the `brand` property directly to the global Window object, and `myCar` is set to `undefined`.

Q4: Name the 4 exact steps that happen behind the scenes when the `new` keyword is executed.

Answer: 1. An empty object `{}` is created. 2. The constructor function is called. 3. The `this` keyword is forcefully bound to the newly created empty object. 4. The fully constructed object is automatically returned.

Q5: How can you programmatically verify if `userOne` was created using the `User` constructor function?

Answer: By using the `instanceof` operator. Writing `console.log(userOne instanceof User);` will return `true` if it was created by that specific blueprint.